

Lab 03 - Commands, Arguments and Environment

Lab id	Lab 03
Day	1 - Cloud Native Foundations & Kubernetes Core Concepts
Duration	40 minutes
Namespace	kcna-lab03
Mapping	LO1 Develop a Kubernetes architectural proof of concept A1 Develop an architectural proof of concept K1 Process for developing proof of concepts

1. Objective

By the end of this lab you will be able to:

- State exactly how `spec.containers[].command` and `spec.containers[].args` map onto a container image's ENTRYPOINT and CMD, and prove it by reading logs.
- Inject configuration with `env`, `envFrom` (including prefix and optional), `configMapKeyRef`, and the downward API (`fieldRef`, `resourceFieldRef`).
- Build ConfigMaps from a real `.env` file and a real JSON file, and mount the JSON into a container.
- Explain where `$(VAR)` expansion happens and what it can and cannot reference.
- Diagnose a `CrashLoopBackOff` from a missing variable using `kubectl logs --previous` and `lastState.terminated.exitCode`.

2. Prerequisites

- Kubernetes v1.30+ cluster; `kubectl` on PATH.
- Labs 01 and 02 completed.

```
kubectl version
```

```
Client Version: v1.37.0
Kustomize Version: v5.8.1
Server Version: v1.37.0
```

```
kubectl get nodes
```

NAME	STATUS	ROLES	AGE	VERSION
kcna-qa-20260905-control-plane	Ready	control-plane	72m	v1.37.0

```
cd courseware/labs/lab-03-commands-args-env
```

3. Scenario

Meridian Freight Pte Ltd runs one container image for the Depot Portal booking API, but it must run in four places: the Singapore West depots, the Johor cross-docks, the staging environment and a developer laptop. Today the team solves this by baking a config.properties into the image and rebuilding it four times - four images, four SHAs, four things to audit.

The PoC has to demonstrate the cloud native answer: one immutable image, configuration supplied at run time. data/route-defaults.env is the settings file the operations team maintains, and data/tariff-rates.json is the pricing table that finance publishes monthly. Neither may ever be baked into an image.

4. Step-by-step procedure

Step 1 - Create the namespace

```
kubectl apply -f manifests/00-namespace.yaml
```

```
namespace/kcna-lab03 created
```

Step 2 - The ENTRYPOINT / CMD rule, before you run anything

Every OCI image can carry two defaults:

- ENTRYPOINT - the executable.
- CMD - the default arguments to that executable (or, if there is no

ENTRYPOINT, the whole command).

Kubernetes overrides them with two **different** field names. This mismatch is the single most common confusion in the KCNA syllabus:

You set in the Pod spec	Container runs	Image ENTRYPOINT	Image CMD
neither command nor args	ENTRYPOINT CMD	used	used
args only	ENTRYPOINT args	used	replaced
command only	command	replaced	discarded
both command and args	command args	replaced	replaced

Memorise the mapping: command → ENTRYPOINT, args → CMD. And note the third row: setting command alone silently throws the image's CMD away.

Confirm the field names against the live schema:

```
kubectl explain pod.spec.containers.command
```

```

GROUP:
KIND:      Pod
VERSION:   v1

FIELD: command <[]string>

DESCRIPTION:
  Entrypoint array. Not executed within a shell. The container image's
  ENTRYPOINT is used if this is not provided. Variable references $(VAR_NAME)
  are expanded using the container's environment.

```

Two facts in that text you must not skip:

1. "Not executed within a shell." command: ["echo hello && echo world"]

does not work; there is no shell to interpret &&. If you need shell syntax you must ask for one explicitly: command: ["/bin/sh", "-c"].

2. "Variable references \$(VAR_NAME) are expanded" - by the kubelet, before the process starts. Not \${VAR}; not backticks. Only \$(VAR_NAME).

Step 3 - Run the override matrix

```
kubectl apply -f manifests/10-entrypoint-matrix.yaml
```

```

pod/em-a-image-defaults created
pod/em-b-args-only created
pod/em-c-command-override created

```

```
kubectl -n kcna-lab03 get pods
```

NAME	READY	STATUS	RESTARTS	AGE
em-a-image-defaults	1/1	Running	0	31s
em-b-args-only	0/1	Completed	0	31s
em-c-command-override	0/1	Completed	0	31s

Case A - neither field set. The image's ENTRYPOINT (/docker-entrypoint.sh) runs with the image's CMD (nginx -g "daemon off;"), so nginx serves and the Pod stays Running:

```
kubectl -n kcna-lab03 logs em-a-image-defaults | head -n 3
```

```

/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform
configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh

```

Case B - args: ["nginx", "-v"], no command. The ENTRYPOINT is untouched; only the arguments changed:

```
kubectl -n kcna-lab03 logs em-b-args-only
```

```

/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform
configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
/docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
/docker-entrypoint.sh: Configuration complete; ready for start up
nginx version: nginx/1.27.5

```

The banner is the ENTRYPOINT; nginx version: is your args running instead of the CMD. The container then exits 0 - hence Completed, because this Pod uses restartPolicy: Never.

Case C - command set. The ENTRYPOINT is gone:

```
kubectl -n kcna-lab03 logs em-c-command-override
```

```

case C: image ENTRYPOINT was replaced by spec.command
case C: pid 1 is now: /bin/sh -c echo "case C: image ENTRYPOINT was replaced by spec.command" .
..
case C: no /docker-entrypoint.sh banner above == proof

```

No banner. That absence is the evidence: /docker-entrypoint.sh never ran, so any configuration work it would have done (IPv6 listen, template substitution, worker tuning) silently did not happen. This is why overriding command on an unfamiliar image is risky - you may be skipping its initialisation.

Inspect what the API stored for each case:

```

kubectl -n kcna-lab03 get pods -o custom-columns=\
'NAME:.metadata.name,COMMAND:.spec.containers[0].command,ARGS:.spec.containers[0].args'

```

NAME	COMMAND	ARGS
em-a-image-defaults	<none>	<none>
em-b-args-only	<none>	[nginx -v]
em-c-command-override	[/bin/sh -c]	[case C: image ENTRYPOINT was replaced by spec. command...]

<none> means "fall back to the image". Kubernetes does not copy the image's ENTRYPOINT into the Pod spec - the runtime resolves it at start-up.

Step 4 - Build a ConfigMap from an env file

```
cat data/route-defaults.env
```

```
# Meridian Freight Pte Ltd - Depot Portal booking-service defaults
# Synthetic dataset for KCNA Lab 03.
# Loaded with: kubectl create configmap route-defaults --from-env-file=<this file>
# Every line must be KEY=VALUE. Blank lines and #-comments are ignored.
DEPOT_REGION=sg-west
DEPOT_CODE=MF-SIN-02
BOOKING_WINDOW_MINUTES=45
MAX_TRAILERS_PER_BAY=3
COLD_CHAIN_REQUIRED=true
CURRENCY=SGD
LOG_LEVEL=info
BOOKING_API_HOST=booking.depot-portal.svc.cluster.local
BOOKING_API_PORT=8080
```

--from-env-file parses KEY=VALUE lines into separate keys. Contrast with --from-file, which makes one key holding the whole file. Preview both:

```
kubectl -n kcna-lab03 create configmap route-defaults \
  --from-env-file=data/route-defaults.env --dry-run=client -o yaml
```

```
apiVersion: v1
data:
  BOOKING_API_HOST: booking.depot-portal.svc.cluster.local
  BOOKING_API_PORT: "8080"
  BOOKING_WINDOW_MINUTES: "45"
  COLD_CHAIN_REQUIRED: "true"
  CURRENCY: SGD
  DEPOT_CODE: MF-SIN-02
  DEPOT_REGION: sg-west
  LOG_LEVEL: info
  MAX_TRAILERS_PER_BAY: "3"
kind: ConfigMap
metadata:
  creationTimestamp: null
  name: route-defaults
```

Nine keys; the comment lines are gone. Note "8080" is quoted - ConfigMap values are always strings. Now create both ConfigMaps for real:

```
kubectl -n kcna-lab03 create configmap route-defaults \
  --from-env-file=data/route-defaults.env
kubectl -n kcna-lab03 create configmap tariff-rates \
  --from-file=data/tariff-rates.json
```

```
configmap/route-defaults created
configmap/tariff-rates created
```

```
kubectl -n kcna-lab03 get configmaps
```

NAME	DATA	AGE
kube-root-ca.crt	1	4m
route-defaults	9	6s
tariff-rates	1	6s

DATA 9 versus DATA 1 is the whole difference between the two flags.

Step 5 - Deploy the configured booking Pod

```
kubectl apply -f manifests/20-booking-config.yaml
```

```
pod/booking-config created
```

```
kubectl -n kcna-lab03 wait --for=condition=Ready pod/booking-config --timeout=90s
```

```
pod/booking-config condition met
```

```
kubectl -n kcna-lab03 logs booking-config
```

```
=== 1. envFrom: every key of ConfigMap route-defaults ===
DEPOT_REGION=sg-west
DEPOT_CODE=MF-SIN-02
BOOKING_WINDOW_MINUTES=45
CURRENCY=SGD
LOG_LEVEL=info
=== 2. envFrom with prefix: MF_ ===
MF_DEPOT_CODE=MF-SIN-02
=== 3. explicit env, incl. downward API ===
SERVICE_NAME=booking
POD_NAME=booking-config
NODE_NAME=kcna-qa-20260905-control-plane
MEM_LIMIT_MI=64
=== 4. dependent env var expanded by the kubelet ===
BOOKING_ENDPOINT=http://booking.kcna-lab03.svc.cluster.local:8080
=== 5. dataset mounted from ConfigMap tariff-rates ===
TARIFF_TABLE=/etc/depot/tariff-rates.json
tariff entries: 10
rate row for MF-SIN-02:
  { "depot_code": "MF-SIN-02", "bay_hour": 51.00, "cold_chain_surcharge": 21.50,
    "after_hours_multiplier": 1.40 },
=== booking-config ready ===
```

Step 6 - Read the five injection mechanisms in the manifest

Open manifests/20-booking-config.yaml alongside the log above.

Mechanism	Manifest excerpt	Result
envFrom + configMapRef	- configMapRef: {name: route-defaults}	all 9 keys become env vars with their original names
envFrom + prefix	- prefix: MF_	the same 9 keys again as MF_DEPOT_CODE, ... - used to keep two config sources from colliding
envFrom + optional: true	name: route-overrides optional: true	the ConfigMap does not exist and the Pod still starts. Without optional, the Pod would hang in CreateContainerConfigError
env + configMapKeyRef	key: MAX_TRAILERS_PER_BAY → name: TRAILERS_PER_BAY	one key, renamed on the way in
env + fieldRef / resourceFieldRef	metadata.name, spec.nodeName, limits.memory with divisor: 1Mi	the downward API - the Pod learns facts about itself without calling the API server

Verify the dependent-variable expansion. Nothing in the ConfigMap contains that URL; the kubelet built it:

```
kubectl -n kcna-lab03 get pod booking-config \
-o jsonpath='{.spec.containers[0].env[?(@.name=="BOOKING_ENDPOINT")].value}{"\n"}'
```

```
http://$(SERVICE_NAME).kcna-lab03.svc.cluster.local:8080
```

The stored spec still contains the literal \$(SERVICE_NAME). Expansion is a run-time behaviour of the kubelet, not a mutation of the object. Compare with what the process sees:

```
kubectl -n kcna-lab03 exec booking-config -- env | grep BOOKING_ENDPOINT
```

```
BOOKING_ENDPOINT=http://booking.kcna-lab03.svc.cluster.local:8080
```

\$(VAR) can only reference variables defined **earlier** in the same env

list. It cannot reference anything that arrived through envFrom. That is

why SERVICE_NAME is an explicit env entry and not a ConfigMap key.

Look at the full environment:

```
kubectl -n kcna-lab03 exec booking-config -- env | sort | head -n 12
```

```
BOOKING_API_HOST=booking.depot-portal.svc.cluster.local
BOOKING_API_PORT=8080
BOOKING_ENDPOINT=http://booking.kcna-lab03.svc.cluster.local:8080
BOOKING_WINDOW_MINUTES=45
COLD_CHAIN_REQUIRED=true
CURRENCY=SGD
DEPOT_CODE=MF-SIN-02
DEPOT_REGION=sg-west
HOME=/
HOSTNAME=booking-config
LOG_LEVEL=info
MAX_TRAILERS_PER_BAY=3
```

And the mounted dataset:

```
kubectl -n kcna-lab03 exec booking-config -- head -n 6 /etc/depot/tariff-rates.json
```

```
{
  "schema": "meridianfreight/tariff-rates/v1",
  "generated": "2026-09-01T00:00:00Z",
  "currency": "SGD",
  "rates": [
    { "depot_code": "MF-SIN-01", "bay_hour": 42.50, "cold_chain_surcharge": 18.00,
      "after_hours_multiplier": 1.35 },
```

Step 7 - Env vars are a snapshot; mounted files are not

Change a value in the ConfigMap:

```
kubectl -n kcna-lab03 patch configmap route-defaults \
--type merge -p '{"data":{"LOG_LEVEL":"debug"}}'
```

```
configmap/route-defaults patched
```

```
kubectl -n kcna-lab03 exec booking-config -- env | grep LOG_LEVEL
```

```
LOG_LEVEL=info
```

Still info. Environment variables are injected once, at container start. They never update. A mounted ConfigMap volume, by contrast, is refreshed by the kubelet (typically within a minute):

```
kubectl -n kcna-lab03 patch configmap tariff-rates \
  --type merge -p '{"data":{"note.txt":"reviewed 2026-09-05"}}'
```

```
configmap/tariff-rates patched
```

Wait up to 60 seconds, then:

```
kubectl -n kcna-lab03 exec booking-config -- ls /etc/depot
```

```
note.txt
tariff-rates.json
```

The new file appeared without restarting the Pod. This asymmetry drives real architecture decisions: secrets and hot-reloadable settings go in volumes; start-up-only settings go in env.

5. Verification

```
chmod +x verification/checks.sh
bash verification/checks.sh
```

The script asserts the namespace; that route-defaults has exactly as many keys as data/route-defaults.env has KEY=VALUE lines and that tariff-rates carries the shipped JSON; all three ENTRYPOINT/CMD cases including the *absence* of the banner in case C; the booking-config log markers for envFrom, prefix, dependent expansion, resourceFieldRef and the 10 tariff rows; the optional: true reference; and the full CrashLoopBackOff evidence chain.

Expected tail:

```
Result: 23 passed, 0 failed
```

Full annotated evidence: [`verification/expected-output.md`](#).

6. Failure injection - one missing variable, endless restarts

manifests/90-booking-missing-env.yaml runs the same preflight script but never sets TARIFF_TABLE and never references route-defaults.

```
kubectl apply -f manifests/90-booking-missing-env.yaml
```

```
pod/booking-missing-env created
```

Watch it for about a minute:

```
kubectl -n kcna-lab03 get pod booking-missing-env -w
```

NAME	READY	STATUS	RESTARTS	AGE
booking-missing-env	0/1	ContainerCreating	0	2s
booking-missing-env	0/1	Error	0	4s
booking-missing-env	0/1	Error	1 (2s ago)	6s
booking-missing-env	0/1	CrashLoopBackOff	1 (5s ago)	11s
booking-missing-env	0/1	Error	2 (16s ago)	27s
booking-missing-env	0/1	CrashLoopBackOff	2 (12s ago)	39s
booking-missing-env	0/1	CrashLoopBackOff	3 (28s ago)	72s

Press Ctrl+C. Try the obvious thing:

```
kubectl -n kcna-lab03 logs booking-missing-env
```

```
Error from server (BadRequest): container "app" in pod "booking-missing-env" is waiting to start: CrashLoopBackOff
```

This is the trap. At the moment you ran logs, the container was not running - it had already exited and the kubelet was waiting out the back-off. The log stream you want belongs to the **previous** instance:

```
kubectl -n kcna-lab03 logs booking-missing-env --previous
```

```
[preflight] booking-api starting
[preflight] FATAL: TARIFF_TABLE is not set - refusing to start
```

There is the cause, in the application's own words. Corroborate with the container status:

```
kubectl -n kcna-lab03 get pod booking-missing-env \
-o jsonpath='{range .status.containerStatuses[0]}{.state.waiting.reason}{"\t"}{.lastState.terminated.exitCode}{"\t"}{.restartCount}{"\n"}{end}'
```

```
CrashLoopBackOff 1 3
```

```
kubectl -n kcna-lab03 describe pod booking-missing-env | sed -n '/Last State/,/Restart Count/p'
```

```
Last State:    Terminated
Reason:       Error
Exit Code:    1
Started:      Sat, 05 Sep 2026 16:03:11 +0800
Finished:     Sat, 05 Sep 2026 16:03:11 +0800
Ready:        False
Restart Count: 3
```

Diagnosis

- ****CrashLoopBackOff is not an error condition; it is a **waiting reason**.**** It

means "this container has exited repeatedly and I am pausing before the next attempt". The back-off doubles - 10s, 20s, 40s, 80s - capped at 5 minutes.

- The real signal is `lastState.terminated.exitCode: 1` plus `Reason:`

Error`**`. Exit 1 is the application deciding to stop. Contrast with exit `**137**` (SIGKILL, usually the OOM killer, Reason: OOMKilled`) and 143 (SIGTERM).

- Started and Finished are the same second - the process died during

start-up, not under load. That points at configuration, not traffic.

- `--previous (-p)` is mandatory for any crash-looping container. Without it

`kubectl logs` targets the container that does not exist yet.

Note what did not happen: the API server accepted the Pod, the scheduler placed it, the image pulled fine. Nothing upstream of the process complained, because nothing upstream knows this application requires `TARIFF_TABLE`.

Repair

A Pod's env is immutable - unlike image, you cannot patch it:

```
kubectl -n kcna-lab03 set env pod/booking-missing-env TARIFF_TABLE=/etc/depot/tariff-rates.json
```

```
error: Pod "booking-missing-env" is invalid: spec: Forbidden: pod updates may not change fields other than `spec.containers[*].image`, `spec.initContainers[*].image`, `spec.activeDeadlineSeconds`, `spec.tolerations` (only additions to existing tolerations), `spec.terminationGracePeriodSeconds` (allow it to be set to 1 if it was previously negative)
```

Read that error carefully - it is the definitive list of what you may change on a running Pod. The fix is to replace the object:

```
kubectl -n kcna-lab03 delete pod booking-missing-env
```

```
pod "booking-missing-env" deleted
```

The corrected workload is already running as `booking-config`. In production, a Deployment would have done this delete-and-recreate for you - which is exactly the argument for not managing bare Pods, and the subject of Lab 07.

Run `bash verification/checks.sh` before deleting the Pod if you want

section 4 of the script to pass rather than skip.

7. Troubleshooting

Symptom	Likely cause	Fix	
CrashLoopBackOff and kubectl logs returns BadRequest ... waiting to start	You are asking for the logs of a container that is not currently running	kubectl logs <pod> --previous (or -p); also read lastState.terminated.exitCode	
Pod stuck in CreateContainerConfigError	envFrom/env references a ConfigMap or Secret that does not exist and is not marked optional: true	kubectl describe pod → Error: configmap "X" not found; create it or add optional: true	
standard_init_linux.go: exec user process caused: exec format error or command not found	command: was given shell syntax (&&, pipes, globs) but is not executed in a shell	Use command: ["/bin/sh", "-c"] and put the script in args	
Container exits immediately with code 0, status Completed	The image CMD is a one-shot program, or your args replaced a long-running CMD	Use a long-running process, or restartPolicy: Never and treat it as a Job (Lab 08)	
Env var is empty inside the container but the key exists in the ConfigMap	Key name mismatch, or the ConfigMap changed after the container started	`kubectl exec <pod> -- env \	sort`; env is a start-up snapshot - recreate the Pod
\$(VAR) appears literally in the process's arguments	The referenced name is not defined earlier in the same env list (e.g. it came from envFrom)	Promote it to an explicit env entry above the one that references it	
Exit code 137 with Reason: OOMKilled	The container exceeded resources.limits.memory	Raise the limit or reduce the workload; see Lab 10	
pod updates may not change fields other than spec.containers[*].image ...	You tried to patch env, command or volumes on a live Pod	Delete and recreate the Pod, or manage it with a Deployment	
Mounted ConfigMap file does not reflect a recent edit	Volume refresh is asynchronous (kubelet sync period, typically under a minute)	Wait, then re-check; if you used subPath, the file will never update - mount the directory instead	

8. Cleanup

```
kubectl delete namespace kcna-lab03
```

```
namespace "kcna-lab03" deleted
```

9. What you learned

- command overrides ENTRYPOINT; args overrides CMD. Setting command

alone also discards the image's CMD - and skips whatever initialisation the image's entrypoint script would have done.

- command/args are not run in a shell. Ask for one explicitly if you need shell syntax.
- `--from-env-file` produces one ConfigMap key per line; `--from-file` produces one key holding the whole file. ConfigMap values are always strings.
- `envFrom` bulk-imports a ConfigMap; prefix prevents collisions; ``optional: true`` stops a missing map from blocking start-up.
- The downward API (`fieldRef`, `resourceFieldRef`) lets a Pod learn its own name, node, IP and resource limits with no API call and no RBAC.
- `$(VAR)` expansion happens in the kubelet, only against earlier entries of the same env list, and the stored object keeps the literal reference.
- Env is a start-up snapshot; volumes are live. That asymmetry decides where each piece of configuration should go.
- `CrashLoopBackOff` is a back-off *state*. The diagnosis lives in `logs --previous, lastState.terminated.exitCode` and `Reason`.

10. Further reading

- [Define a Command and Arguments for a Container](#)
- [Define Environment Variables for a Container](#)
- [Define Dependent Environment Variables](#)
- [ConfigMaps](#)
- [Configure a Pod to Use a ConfigMap](#)
- [Expose Pod Information to Containers Through Environment Variables \(downward API\)](#)
- [Pod Lifecycle - container states and restart back-off](#)
- [Debug Running Pods](#)